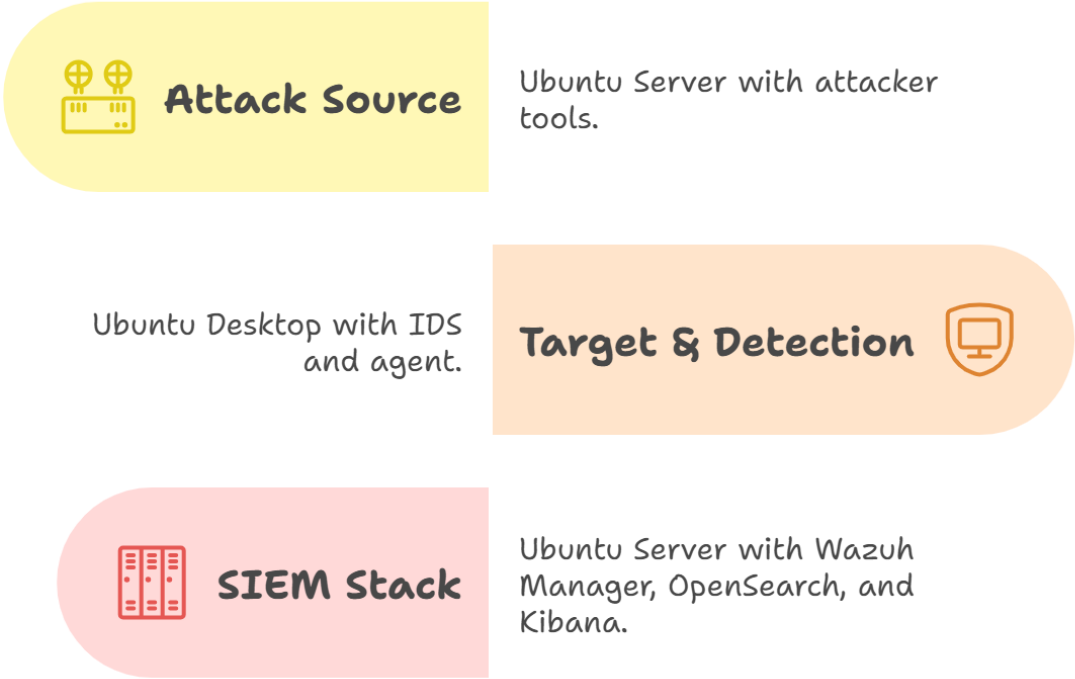
# Wazuh SIEM : Dashboard Kibana & Détection Brute-Force SSH




Réalisé par : TINGO Mik

Date : Mai 2026

Outils : Wazuh 4.14 · Kibana · Hydra · Nmap

Email : tinigojeanmikcomlan@gmail.com

LinkedIn : http://www.linkedin.com/in/jean-mik-c-tinigo

# SOMMAIRE

GLOSSAIRE ..... 3
CONTEXTE ET OBJECTIFS ..... 4
ARCHITECTURE DU LABORATOIRE ..... 5
ENVIRONNEMENT MATERIEL ET LOGICIEL ..... 5
SCHEMA RESEAU ET SCHEMA D'ATTAQUE ..... 5
MISE EN PLACE DU DASHBOARD ..... 7
GÉNÉRATION DES DONNÉES D'ATTAQUE ..... 7
LES 4 VISUALISATIONS ..... 9
TOP 10 ALERT TYPES (BAR CHART) ..... 12
ALERT TIMELINE (LINE CHART) ..... 13
ALERT SOURCES DISTRIBUTION (DONUT CHART) ..... 14
TOP ATTACKER IPS (DATA TABLE) ..... 15
ASSEMBLAGE DU DASHBOARD ..... 16
RÈGLE CUSTOM WAZUH - DÉTECTION BRUTE-FORCE SSH ..... 19
POURQUOI UNE REGLE COMPORTEMENTALE ? ..... 19
LA REGLE XML ..... 19
POURQUOI IF_MATCHED _SID 5760 ? ..... 20
RESULTAT ..... 21
BILAN DES RÈGLES ET VISUALISATIONS ..... 22
CONCLUSION ET AXES D'AMELIORATION ..... 23

# GLOSSAIRE

- **Dashboard :** Interface graphique regroupant plusieurs visualisations (graphiques, jauges, tableaux) sur une seule page. Il permet une supervision en temps réel des alertes, de l'état des agents et des métriques de sécurité**.**
- **Visualisation :** Représentation graphique d'une donnée issue d'un index (ex : diagramme en barres des 10 alertes les plus fréquentes)
- **Kibana / OpenSearch Dashboards :** Ce sont les interfaces web de visualisation et d'exploration des données stockées dans OpenSearch (moteur de recherche).
  **Kibana** est une interface historique utilisée avec Elasticsearch.
  **OpenSearch Dashboard** est une version compatible avec OpenSearch (utilisée par Wazuh depuis la version 4.x).
  Les deux permettent : requêter les logs (Discover), créer des visualisations (Visualize), construire des dashboards, et gérer les index.
- **Rule level** : Dans Wazuh, chaque règle de détection possède un **niveau de criticité** de 0 à 15. Il est utilisé pour prioriser les alertes. Un agent SOC paramètre souvent des actions automatiques (ex : blocage d'IP) à partir du niveau 10
- **Brute-force :** Une attaque par force brute consiste à essayer de nombreuses combinaisons (identifiants / mots de passe) pour accéder à un service (SSH, WordPress, RDP, etc.).
- **SIEM (Security Information and Event Management) :** est une solution logicielle qui centralise, normalise et analyse en temps réel les données de sécurité provenant de l'ensemble du système d'information, permettant ainsi la détection des menaces, l'investigation et la conformité.
- **Threshold :** Seuil à atteindre.
- **Frequency :** Seuil + fenêtre de temps
- **DQL (Dashboard Query Language) :** DQL est le langage de requête utilisé par OpenSearch Dashboards pour filtrer et rechercher des données dans l'interface Discover.

# CONTEXTE ET OBJECTIFS

## Contexte

Dans le cadre de ma préparation au métier d'**Analyste SOC**, à la suite du projet 2.1 (Wazuh déployé, Suricata intégré) j'ai choisi de rajouter une interface de visualisation des alertes en temps réel car un SIEM sans dashboard n'a pas de valeur opérationnelle. Le but est donc de construire la couche de visualisation et de rajouter une règle de détection comportementale custom.

## Objectifs du projet

Ce projet vise à :

- Créer 4 visualisations Kibana sur les données Wazuh
- Assembler un dashboard SOC fonctionnel
- Écrire une règle Wazuh custom de détection brute-force
- Valider par simulation d'attaque réelle (Hydra)

## Compétences visées

- Configuration Kibana (agrégations, visualisations)
- Lecture et interprétation d'un dashboard SOC
- Rédaction de règles Wazuh (XML, frequency rules)
- Corrélation comportementale (vs signature)

# ARCHITECTURE DU LABORATOIRE

## ENVIRONNEMENT MATERIEL ET LOGICIEL

| Component | Role | OS | IP | Tools |
|---|---|---|---|---|
| **Ubuntu VM** | Target + IDS + Wazuh agent | Ubuntu Desktop 24.04 | 10.186.252.127 | Kibana + Wazuh rules |
| **Ubuntu server VM** | Attacker + Wazuh server | Ubuntu Server 22.04 | 10.186.252.114 | Hydra + Nmap |
| **Hypervisor** | Host | Oracle VirtualBox 7.2.4 | - | - |

Les deux machines communiquent via un réseau interne sur VirtualBox ***Accès par pont***.

Le système Ubuntu Desktop héberge Suricata (IDS) et l'agent-wazuh.

Le système Ubuntu Server héberge le manager Wazuh (SIEM) pour la gestion des alertes.

## SCHEMA RESEAU ET SCHEMA D'ATTAQUE

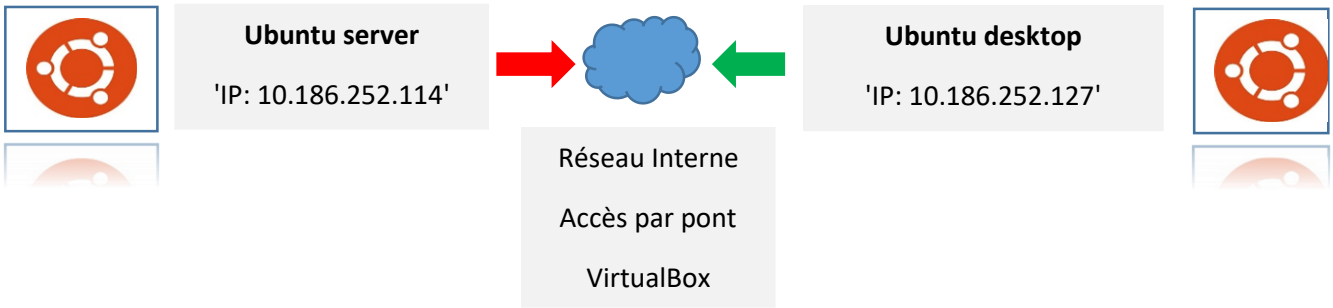



**Figure 1** : Architecture du Labo

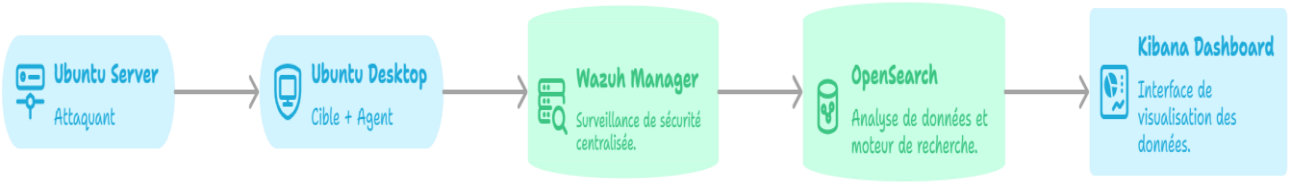



**Figure 2** : Schéma de l'attaque

# MISE EN PLACE DU DASHBOARD

## GÉNÉRATION DES DONNÉES D'ATTAQUE

La mise en place du dashboard nécessite une certaine quantité de données pour une visualisation optimale. Il nous faut donc générer le trafic, c'est-à-dire les données d'alertes qui seront utilisés pour nourrir notre dashboard car sans données à afficher, le dashboard est inutile.

Voici la liste de commandes utilisés pour générés les données d'attaques :

```
ping -c 50 10.186.252.127

nmap -sS -T4 -sV -O 10.186.252.127
```

Le **ping** sert à effectuer un test de connectivité entre deux machines. Cela s'apparente à du scan réseau dans la contexte d'une attaque.
L'option **-c** permet de définir le nombre de paquets que l'on souhaite envoyés vers la machine cible, cinquante paquets (50) dans notre cas.

Le **nmap** est un outil open-source qui permet d'explorer un réseau, d'identifier les machines connectées, les ports ouverts, les services en cours d'exécution et les systèmes d'exploitation.
Les options :

| Option | Nom | Explication |
|---|---|---|
| **sS** | **SYN scan (half-open)** | Scan "furtif" : envoie un SYN, reçoit SYN-ACK (port ouvert) ou RST (fermé), mais **n'achève pas** le handshake TCP. Moins détectable par les logs applicatifs. |
| **T4** | **Timing template 4 (aggressive)** | Accélère le scan. -T0 = très lent (furtif), -T5 = très agressif. -T4 suppose un réseau rapide (LAN). |
| **sV** | **Version detection** | Interroge les ports ouverts pour identifier la **version exacte** du service (ex: Apache httpd 2.4.52 au lieu de juste http) |

| | | |
|---|---|---|
| **O** | **OS detection** | Tente d'**identifier le système d'exploitation** de la cible (ex: Linux 5.x, Windows 10). Nécessite des privilèges root/sudo. |

```
echo -e "admin\nroot\ntest\npassword\n123456\nwelcome\nletmein" > ~/passwords.txt

hydra -l root -P ./passwords.txt ssh://10.218.45.127 -t 4 -V
```

La commande **echo** affiche une un ensemble de mots en prenant en compte les caractères d'échappement **/n* qui permet le retour à la ligne grace à l'option **-e*. La sortie de la commande est ensuite redirigée *>* vers le fichier **passwords.txt*, il y écrase le contenu si le fichier existe déjà. On obtient au final une wordlist qui nous servira de bibliothèque de référence pour tester une palette de mots de passe.
La commande **hydra* permet d'effectuer une attaque par brute-force SSH.

Détail des options :

| Option | Signification | Explication |
|---|---|---|
| **-l root** | Login (username) | Teste avec le compte root |
| **-P ./passwords.txt** | Password File | Utilise la liste de mots de passe du fichier |
| **ssh://10.218.45.127** | Cible | Cible le service SSH sur l'IP 10.218.45.127 |
| **-t 4** | Threads | Exécute 4 tentatives en parallèle |
| **-V** | Verbose | Affiche chaque tentative en temps réel |

Il connecte au service SSH sur 10.218.45.127, et pour chaque mot de passe dans passwords.txt, tente une authentification avec l'utilisateur root, affiche les résultats (succès ou échec) et continue jusqu'à épuisement de la liste ou découverte du mot de passe.

```
hackcat@hackcat77vm:~$ hydra -l root -P ./passwords.txt ssh://10.186.252.127 -t 4 -V
Hydra v9.2 (c) 2021 by van Hauser/THC & David Maciejak - Please do not use in military or secret ser
vice organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics a
nyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-09 21:43:00
[DATA] max 4 tasks per 1 server, overall 4 tasks, 7 login tries (l:1/p:7), ~2 tries per task
[DATA] attacking ssh://10.186.252.127:22/
[ATTEMPT] target 10.186.252.127 - login "root" - pass "admin" - 1 of 7 [child 0] (0/0)
[ATTEMPT] target 10.186.252.127 - login "root" - pass "root" - 2 of 7 [child 1] (0/0)
[ATTEMPT] target 10.186.252.127 - login "root" - pass "test" - 3 of 7 [child 2] (0/0)
[ATTEMPT] target 10.186.252.127 - login "root" - pass "password" - 4 of 7 [child 3] (0/0)
[ATTEMPT] target 10.186.252.127 - login "root" - pass "123456" - 5 of 7 [child 0] (0/0)
[ATTEMPT] target 10.186.252.127 - login "root" - pass "welcome" - 6 of 7 [child 1] (0/0)
[ATTEMPT] target 10.186.252.127 - login "root" - pass "letmein" - 7 of 7 [child 2] (0/0)
1 of 1 target completed, 0 valid password found
[WARNING] Writing restore file because 1 final worker threads did not complete until end.
[ERROR] 1 target did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-09 21:43:07
hackcat@hackcat77vm:~$ _
```

**Figure 3** : Terminal Hydra en cours d'éxécution

## LES 4 VISUALISATIONS

Nous allons créer quatre visualisations dont la mise en commun formera notre dashboard sur Wazuh.

Se connecter à Wazuh, aller dans la barre de navigation, sélectionner la section ***Explore*** et cliquer sur l'option dashboard

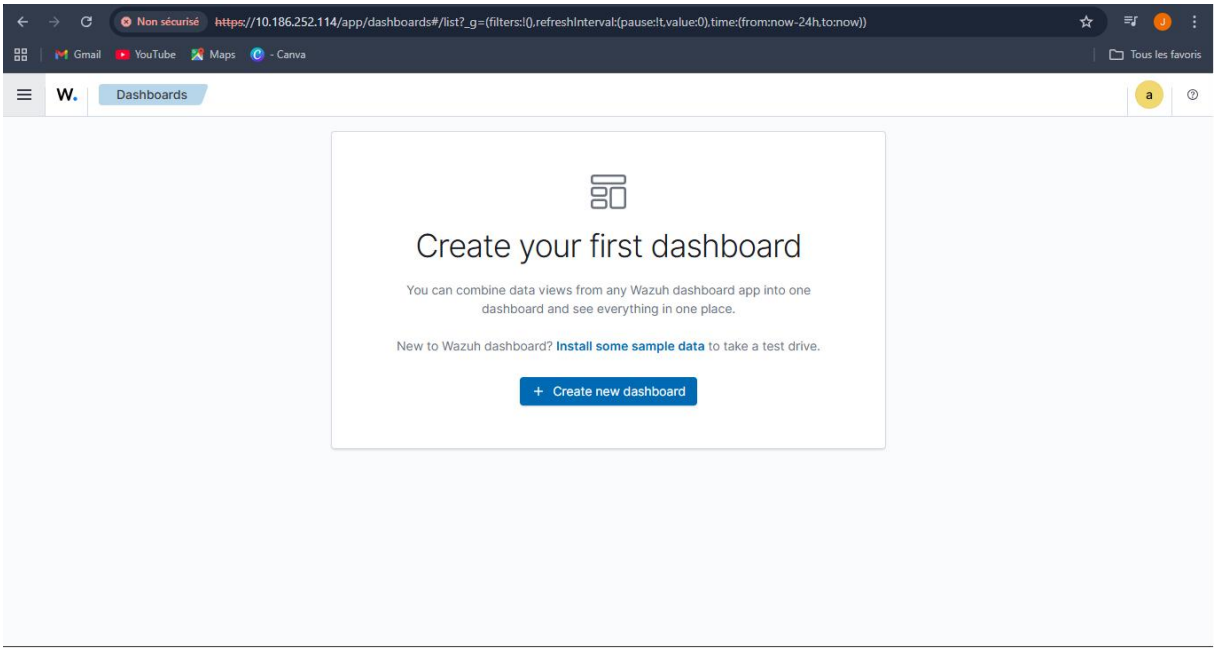



**Figure 4** : Création d'un dashboard

Cliquer sur *+ ***Create new dashboard**** et remplir les champs associées

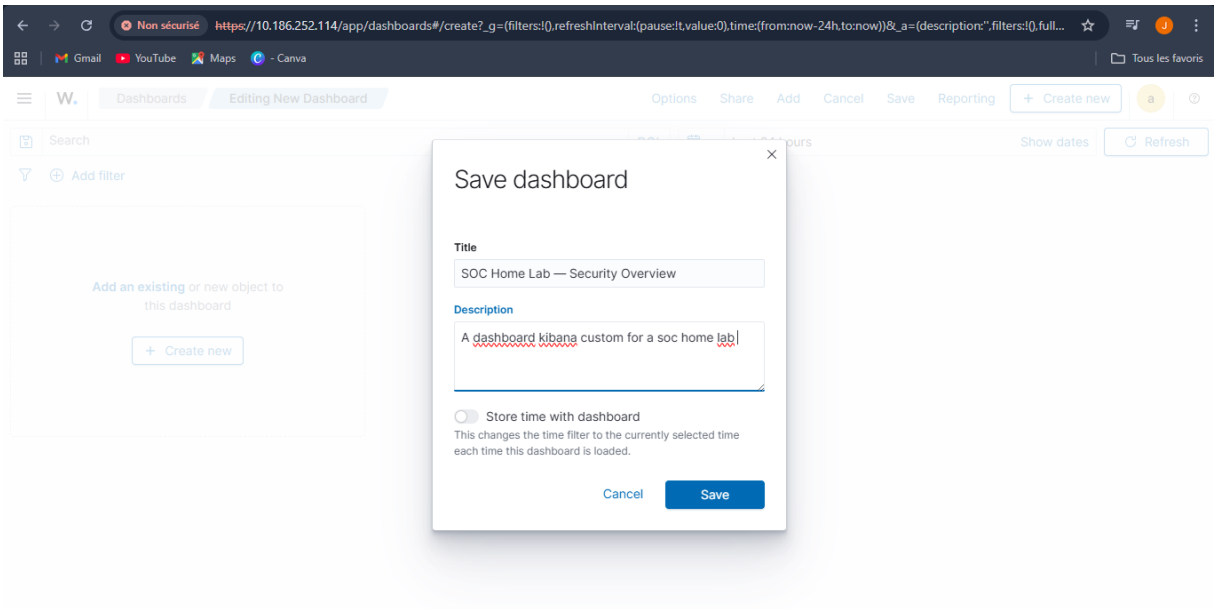



**Figure 5** : Création d'un dashboard

Puis appuyer sur ***Save*** pour sauvegarder.
On accède ensuite au dashboard que l'on vient de crèer et on lance la conception des visualisations.

Cliquer sur ***Add*** , ***Create new***, ***visualisation****

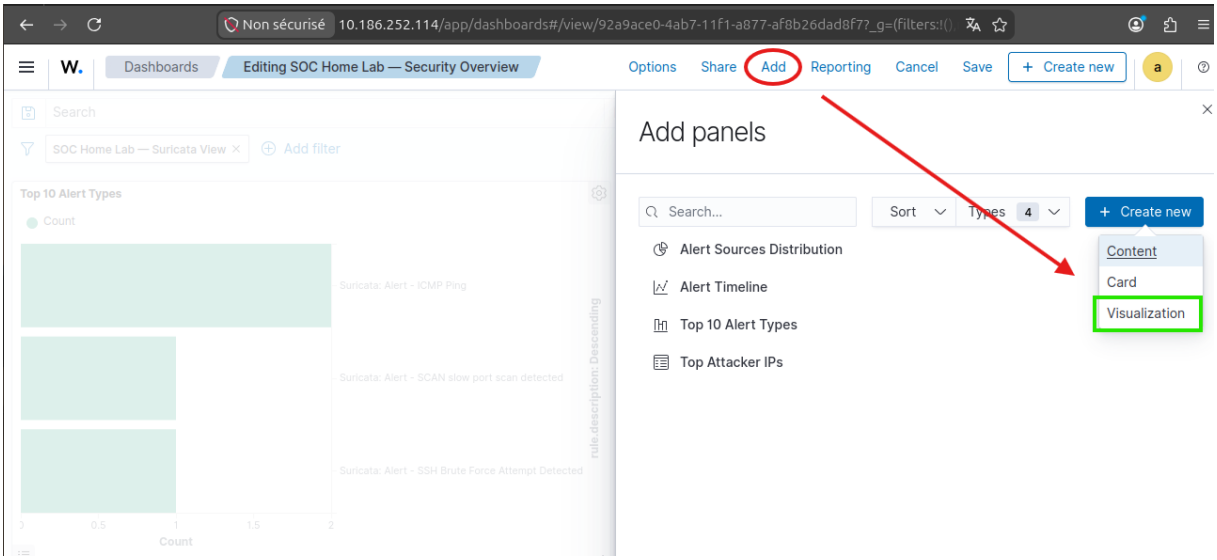



**Figure 6** : Création d'une visualisation

On passe ensuite à la configuration de la visualisation. La première étape consiste à sélectionner le type de la visualisation

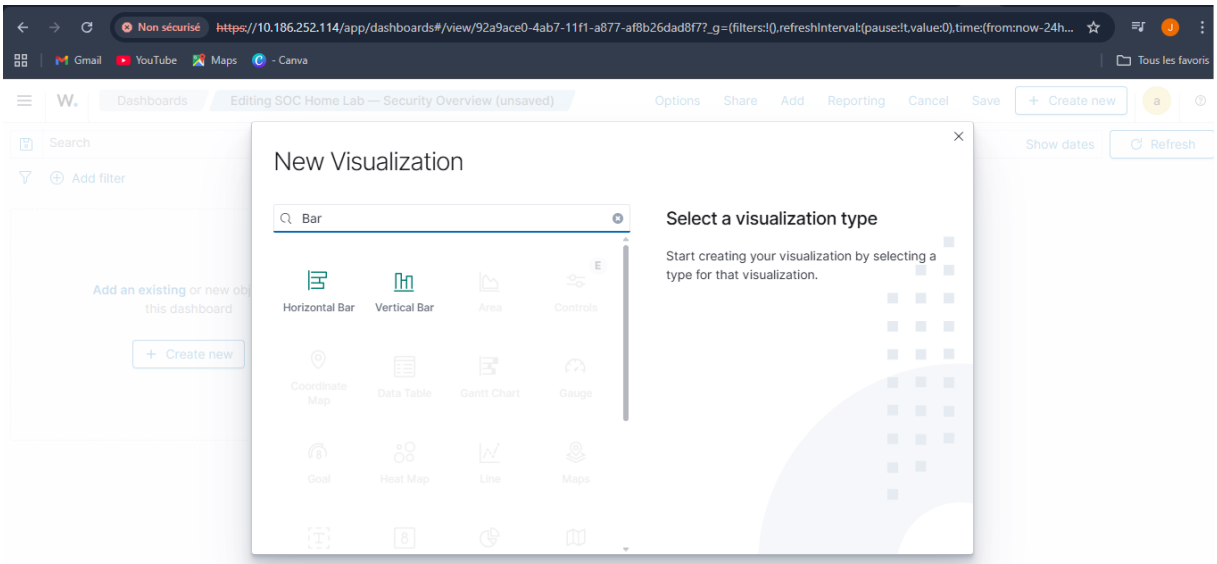



**Figure 7** : Type de la visualisation

La seconde étape consiste à choisir la source de données de notre visualisation. Dans notre cas, nos quatre visualisations auront comme source ***wazuh-alerts-****

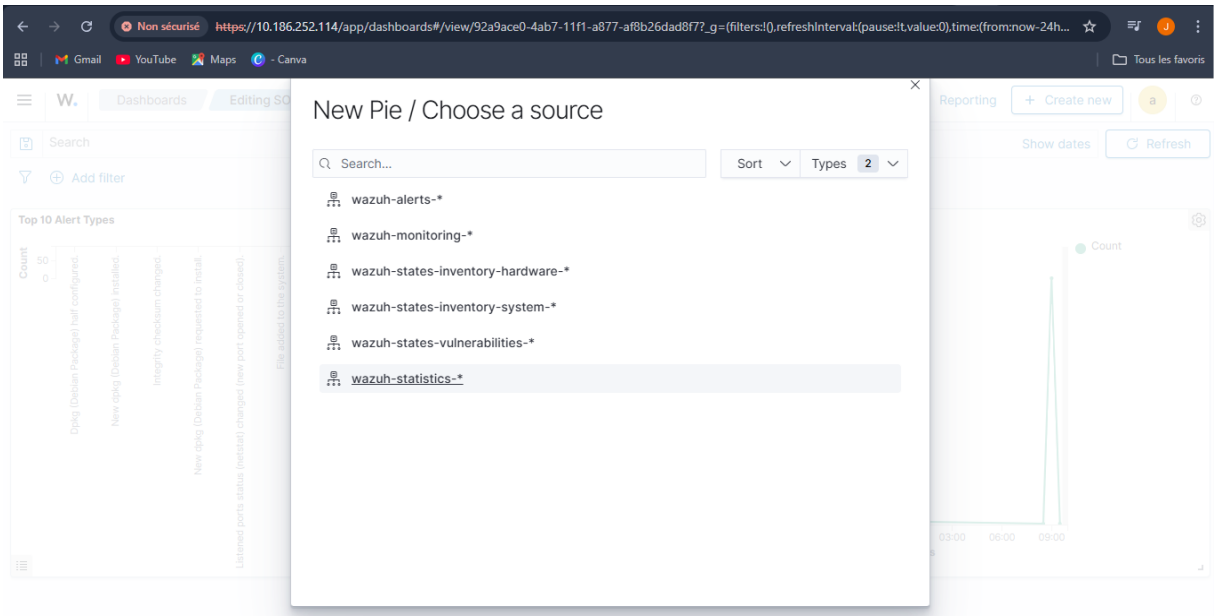



**Figure 8** : Source de données de la visualisation

La troisième étape consiste à sélectionner l'ensemble des options de notre visualisation, nous y reviendrons dans la section suivante.

La derniére étape consiste à enregistrer les configurations éffectués

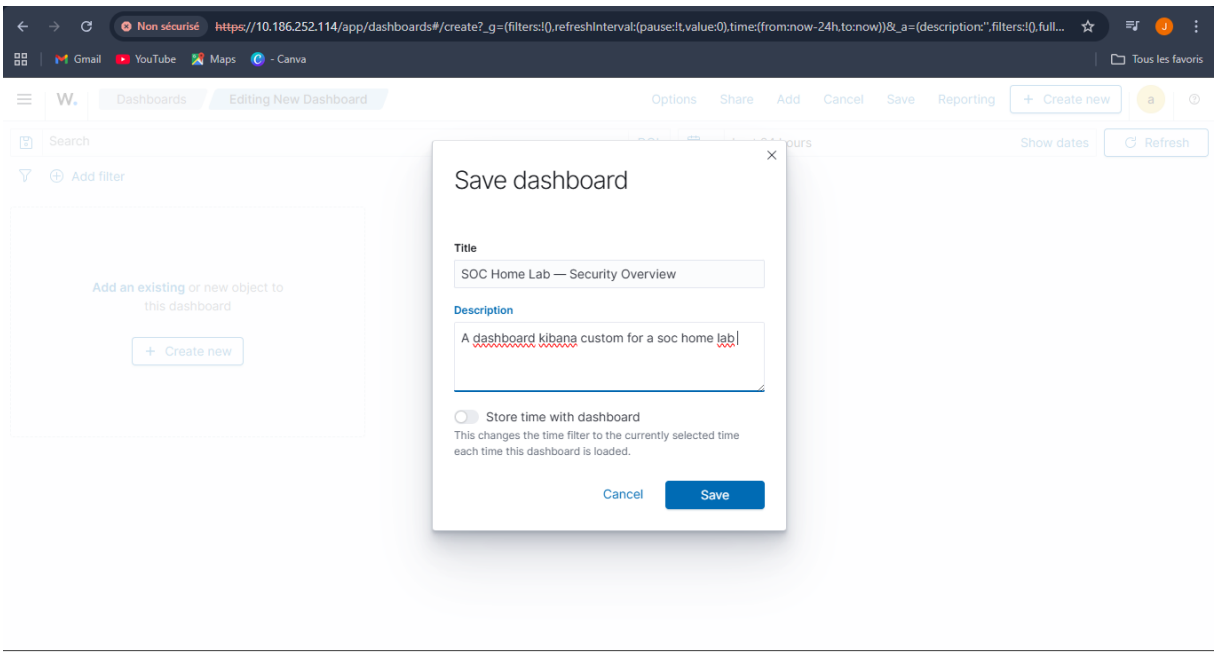



**Figure 9** : Sauvegarde de la visualisation

On en revient à la troisième étape de la configuration, chaque visualisation a une configuration qui lui est propre.

## TOP 10 ALERT TYPES (BAR CHART)

### Configuration

- Visualisation Type : Bar vertical
- Metrics :
  - Y-axis → Count
- Buckets :
  - X-axis → Terms
  - Field → rule.description
  - Order → Descending
  - Size → 10
  - Order by → Metric : Count
- Metrics & Axes :
  - Metrics :

        - Value-axis : BottomAxis-1
        - Mode : Stacked
        - Chart type : Bar
    - Y-axes → BottomAxis-1 Count
    - X-axis :
        - Position : Right
- Save : "Top 10 Alert Types"

**Interprétation :** Sur la visualisation on remarque que les dix (10) alertes les plus fréquentes sont triées de par ordre décroissant. On remarque également que l'alerte la plus fréquente est ****sshd authentication failed***, cela est due aux tentatives de brute force ssh exécutées avec ****hydra*** depuis la machine Ubuntu serveur

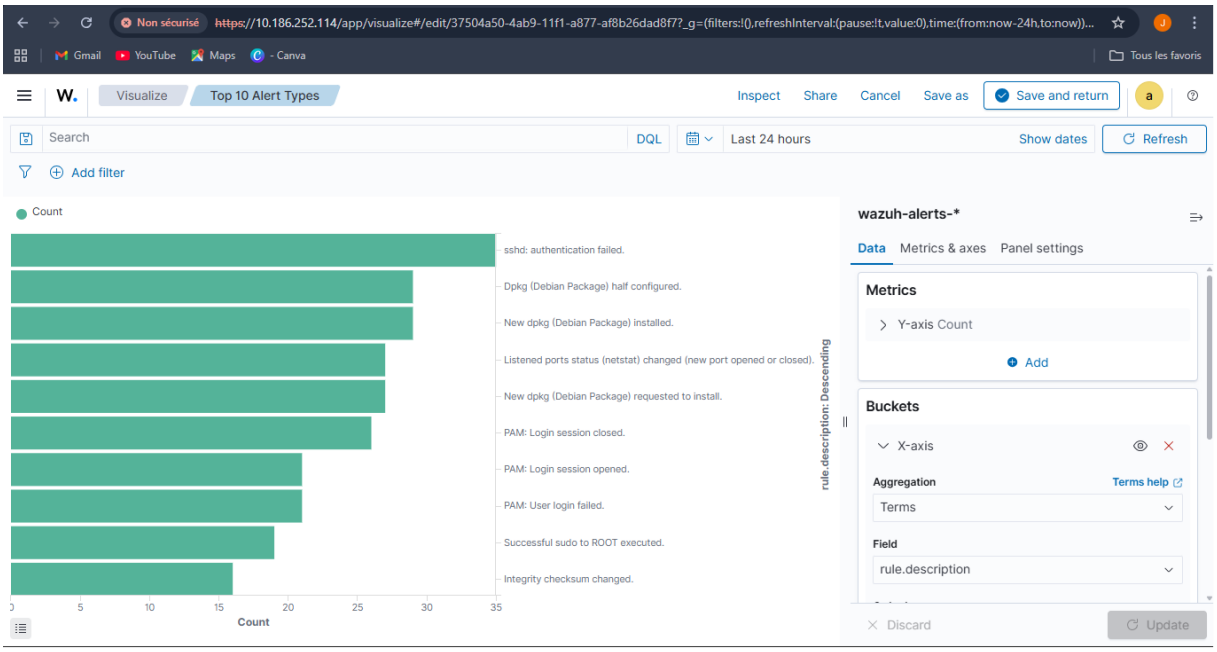



**Figure 10** : Visualisation top 10 alert types

## ALERT TIMELINE (LINE CHART)

### Configuration

- Visualisation Type : Line
- Metrics :
    - Y-axis → Count

- Buckets :
  - X-axis →Date Histogram
  - Field → timestamp
  - Interval → Auto
- Save : "Alert Timeline"

**Interprétation :** On remarque un pic à 140 alertes à 09h00, il s'agit du brute-force de l'attaque avec Hydra, identifiable visuellement. En l'absence de ce contexte, un analyste SOC L1 aurait dû investiguer ce spike immédiatement. Dans le SOC : cela permet de détecter des anomalies temporelles.

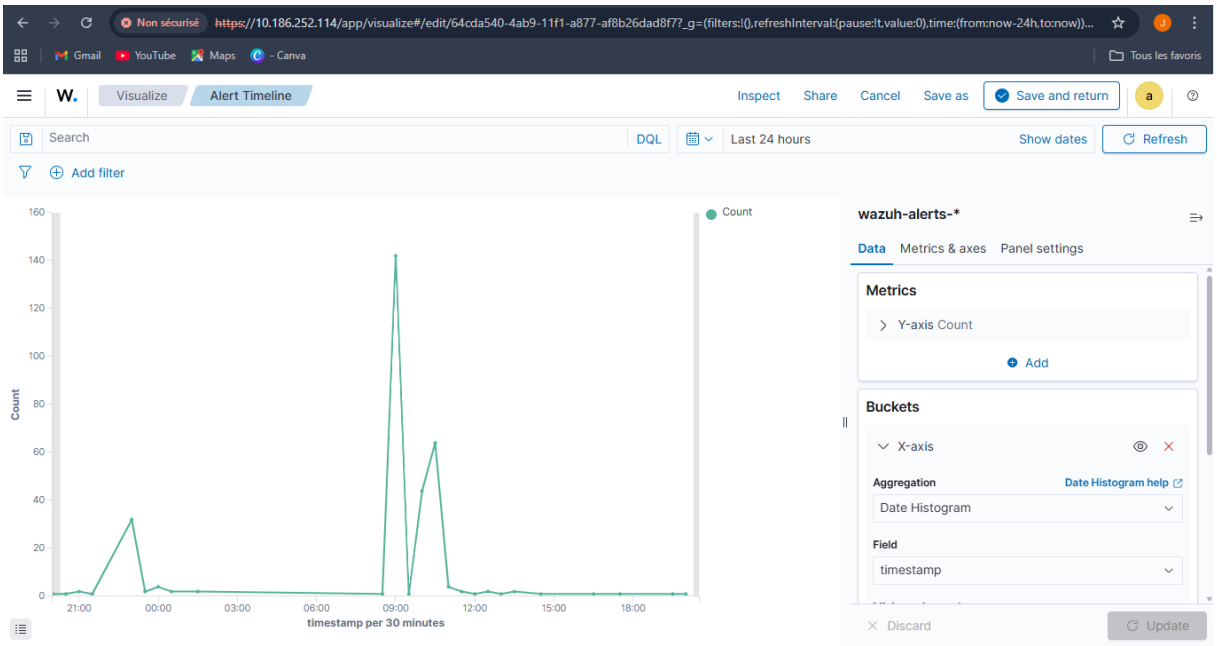



**Figure 11** : Visualisation alert timeline

## ALERT SOURCES DISTRIBUTION (DONUT CHART)

### Configuration

- Visualisation Type : Pie
- Metrics :
  - Slice size → Count
- Buckets :
  - Split slices → Terms
  - Field → rule.groups
  - Size → 5

- Save : "Alert Sources Distribution"

**Interprétation :** On remarque que la source dominante est syslog suivi de dkpg puis ossec, pam, config_changed. En SOC cela montre la répartition des sources de logs.

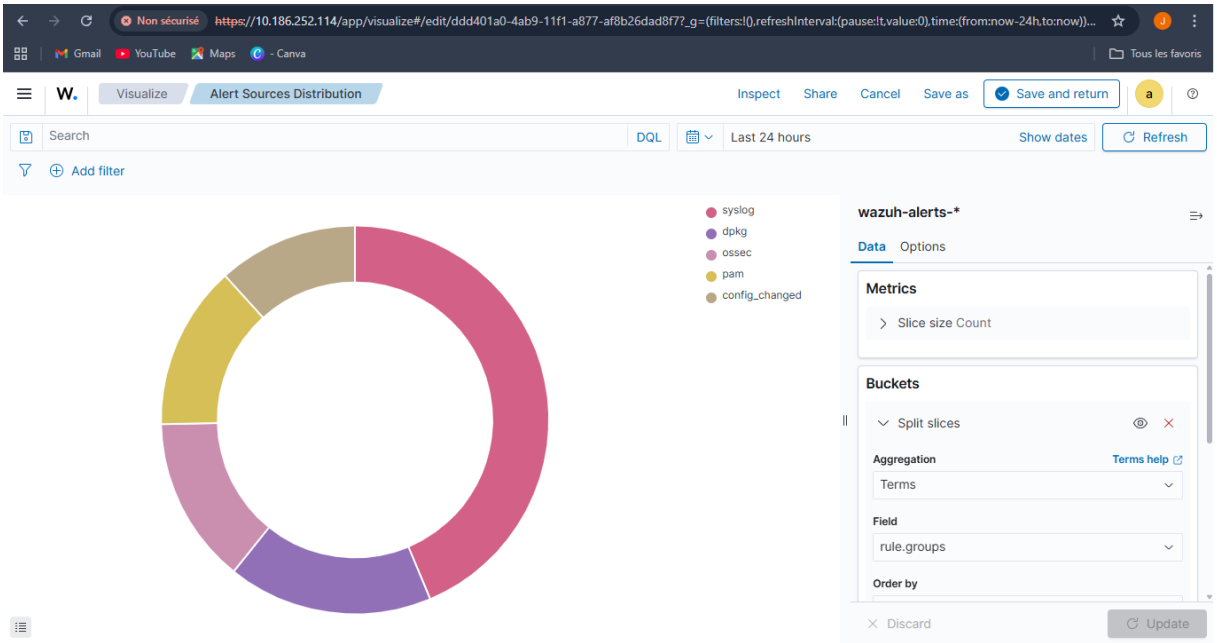



**Figure 12** : Visualisation alert sources distribution

## TOP ATTACKER IPS (DATA TABLE)

### Configuration

- Visualisation Type : Data table
- Metrics : Count
- Buckets :
  - Split rows → Terms
  - Field → data.src_ip
  - Size → 10
- Save : "Top Attacker IPs"

**Interprétation :** Les adresses IP identifiés sont : 10.186.252.114 = attaquant (22 events) 10.186.252.127 = victime (3 events côté source). En SOC, cela permet l'identification immédiate de la source hostile.

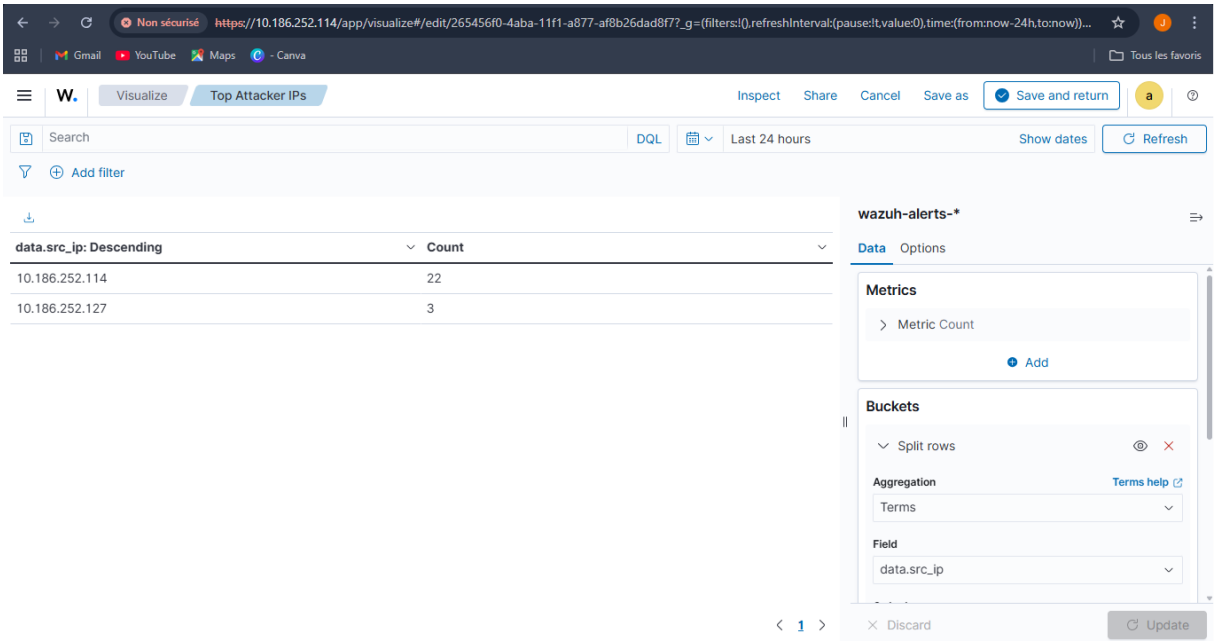



**Figure 13** : Visualisation top attacker ips

## ASSEMBLAGE DU DASHBOARD

Une fois l'ensemble des visualisations crées et sauvegardés, on peut avoir un point de vue de notre dashboard

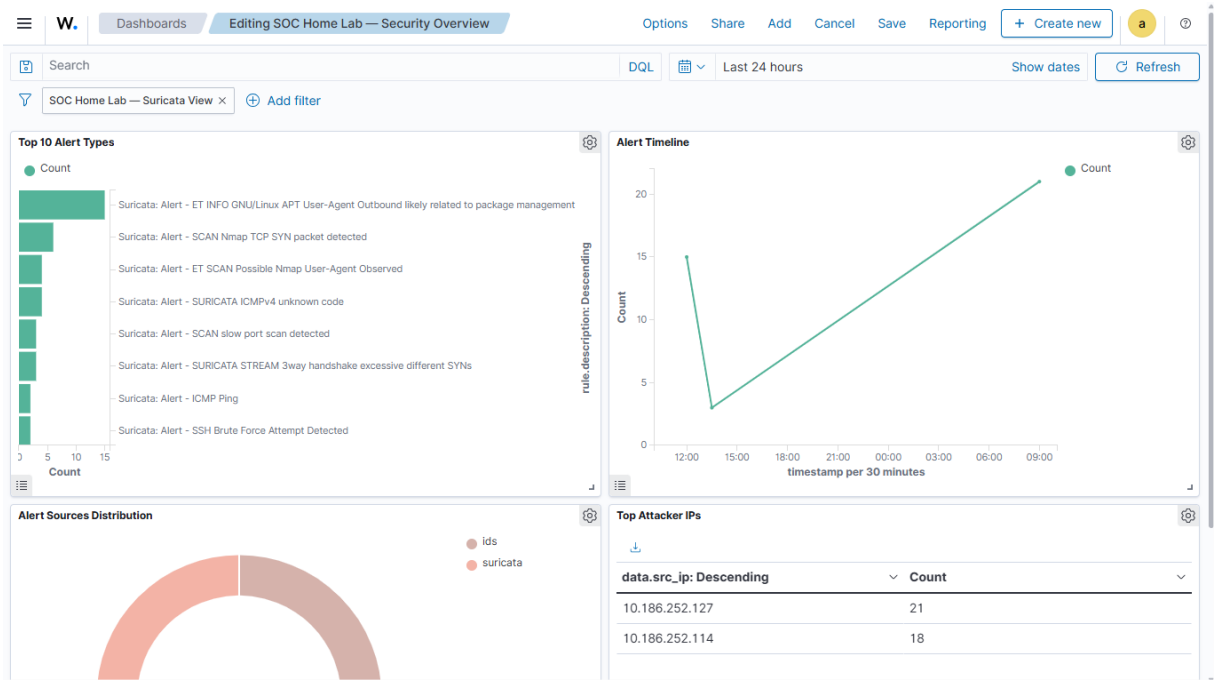
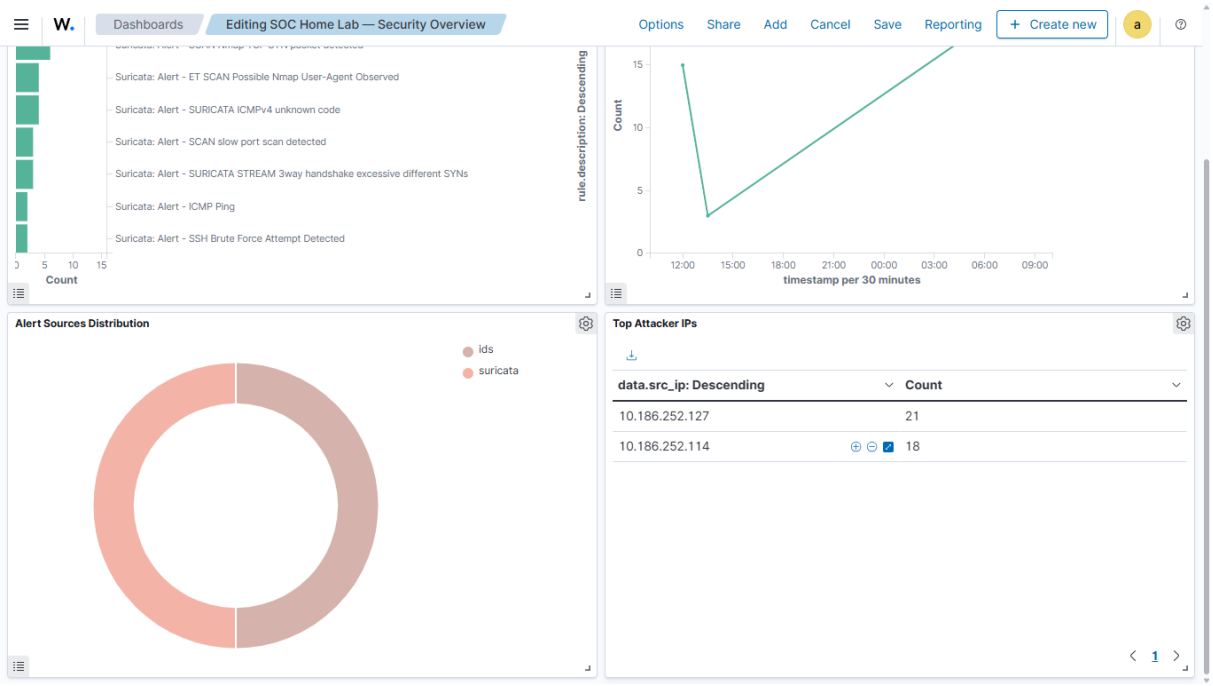



**Figure 13-a** : Dashboard premiere partie




**<u>Figure 13-b</u>** : Dashboard deuxième partie

Sur la figure **13-a** on remarque en haut à gauche la présence du filtre nommé ***SOC Home Lab – Suricata View*** appliqué au dashboard. Il s'agit d'un filtre qui permet de sélectionner les sources de données à afficher, ici on affiche uniquement les logs de Suricata.

Pour ce faire on clique sur ***Add filter***, on remplit les champs comme suit et on enregistre comme "***SOC Home Lab — Suricata View***"

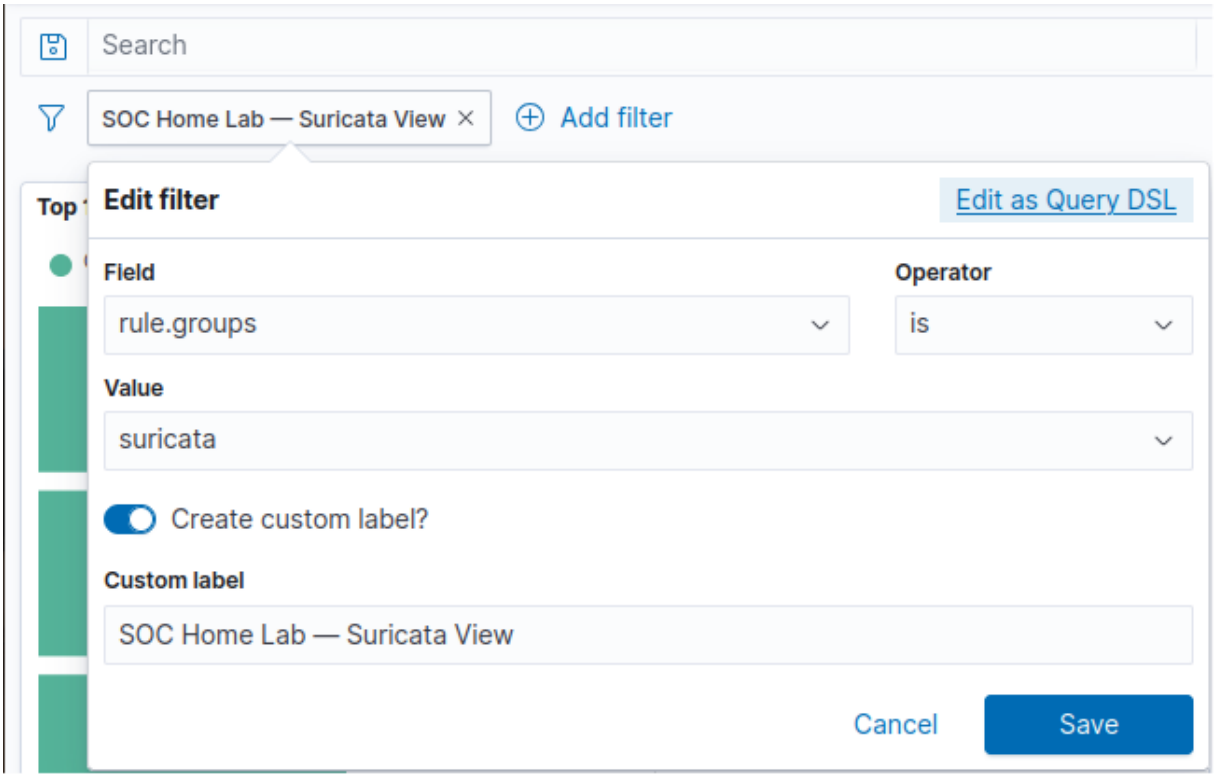



**Figure 14** : Ajout du filtre pour Suricata

Nous possédons maintenant deux vues : une globale (tous les logs) et une Suricata seule. C'est une méthode utilisée en SOC pour séparer les sources.

# RÈGLE CUSTOM WAZUH - DÉTECTION BRUTE-FORCE SSH

## POURQUOI UNE REGLE COMPORTEMENTALE ?

Wazuh a des règles natives pour le brute-force SSH mais face au SSH chiffré, il n'y a pas d'inspection de contenu possible donc seule l'analyse comportementale (fréquence) fonctionne. Ce raisonnement est la meme que celle utilisée pour Suricata au cours de Mois 1 (threshold).

Une maitrise de la syntaxe des règles d'un SIEM est également nécessaire pour un analyste SOC.

## LA REGLE XML

Les règles utilisées sont sous le format XML (e**X**tensible **M**arkup **L**anguage), c'est un langage de balisage standardisé pour structurer des données sous forme textuelle. Extensible et strict, il est utilisé dans les fichiers de configuration (ex: ossec.conf de Wazuh), les échanges machine-à-machine (SOAP) et les documents bureautiques.

Nous allons ouvrir le fichier de configuration local avec la commande ****sudo nano /var/ossec/etc/rules/local_rules.xml**** sur Ubuntu server 22.04 et rajouter la règle XML custom suivante pour la détection brute-force SSH

```
<group name="ssh,brute_force,custom">
        <!-- Détecte 5 échecs SSH en moins de 120 secondes -->
        <rule id="100002" level="10" frequency="5" timeframe="120">
                <if_matched_sid>5760</if_matched_sid>
                 <description>SSH Brute Force: Multiple failed attempts detected</description>
                <mitre>
                    <id>T1110.001</id>
                 </mitre>
                <group>authentication_failures</group>
        </rule>
</group>
```

Cette règle **détecte une attaque par force brute (brute force) sur le service SSH**. Elle se déclenche lorsqu'elle enregistre **5 échecs d'authentification SSH** (c'est-à-dire 5 tentatives avec des mots de passe incorrects) provenant de la même source, et ce, dans une fenêtre de **120 secondes** (2 minutes).

Elle augmente alors le niveau de criticité de l'alerte pour signaler une activité malveillante probable.

| Champ | Valeur | Signification |
|---|---|---|
| **rule_id** | 100002 | C'est l'identifiant de notre règle |
| **level** | 10 | Le niveau de criticité de l'alerte |
| **frequency** | 5 | Seuil de tentatives |
| **timeframe** | 120 | Fenêtre de temps |
| **if_matched_sid** | 5760 | Se base sur les logs *sshd :authentication failed* |
| **id** | T1110.001 | Technique MITRE correspondant |

Afin que les modifications soient prises en compte par wazuh, on lance la commande suivante * ***sudo systemctl restart wazuh-manager***.

## POURQUOI IF_MATCHED _SID 5760 ?

Notre règle de détection se base sur les logs ****sshd : authentication failed***, il fallait donc utiliser le **rule_id**, c'est-à-dire l'identifiant de cette alerte pour déclencher notre règle custom.

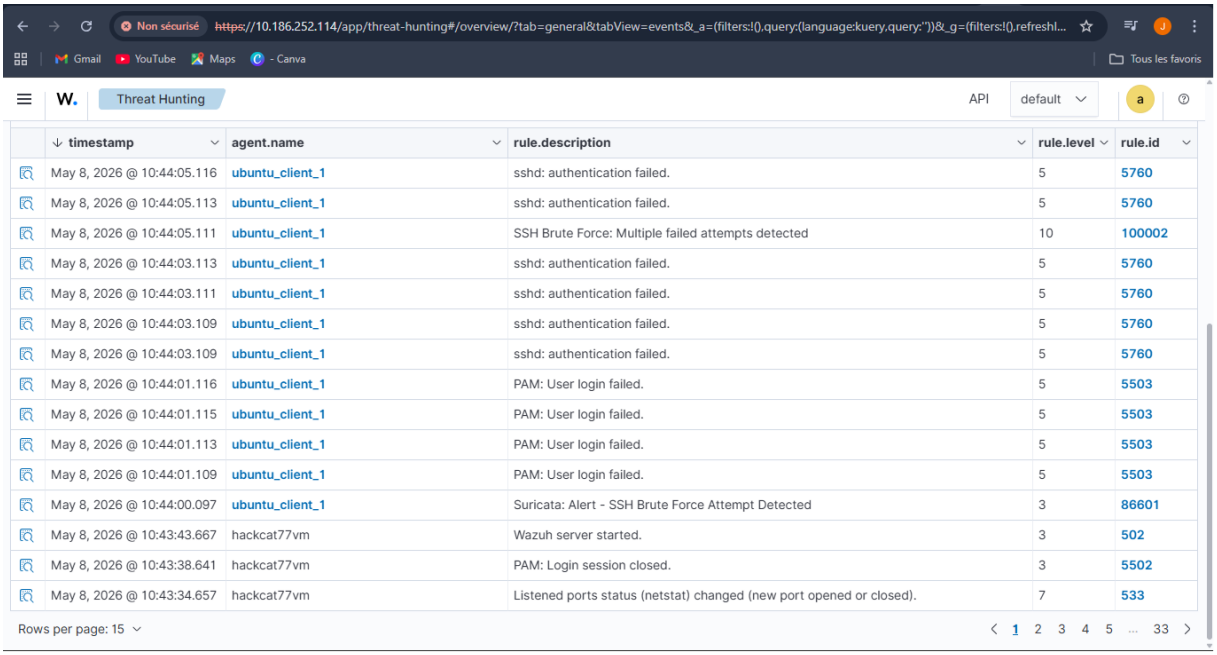
Suite à mes recherches, j'avais utilisé dans un premier temps le SID 5710 mais la règle ne se déclenchais pas malgré les tests de brute-force que j'exécutais. En analysant les alertes du dashboard dans la section ****Threat Hunting*** j'ai constaté que les logs visés c'est-à-dire * ***sshd : authentication failed*** * apparaissaient mais pas celui de ma règle custom. En y jetant un œil de plus près, j'ai remarqué que le *rule_id* affiché dans le dashboard ne correspondait pas à celui utilisé dans ma règle custom.

Le SID réel était 5760, une fois corrigé en modifiant le XML, j'ai redémarrer le wazuh-manager et relancer le test avec ****hydra*** mon alerte se déclencha

directement. J'en ai déduit que le SID est différent en fonction de l'environnement.

La leçon : toujours vérifier les SIDs réels dans son environnement.

## RESULTAT




| timestamp | agent.name | rule.description | rule.level | rule.id |
|---|---|---|---|---|
| May 8, 2026 @ 10:44:05.116 | ubuntu_client_1 | sshd: authentication failed. | 5 | 5760 |
| May 8, 2026 @ 10:44:05.113 | ubuntu_client_1 | sshd: authentication failed. | 5 | 5760 |
| May 8, 2026 @ 10:44:05.111 | ubuntu_client_1 | SSH Brute Force: Multiple failed attempts detected | 10 | 100002 |
| May 8, 2026 @ 10:44:03.113 | ubuntu_client_1 | sshd: authentication failed. | 5 | 5760 |
| May 8, 2026 @ 10:44:03.111 | ubuntu_client_1 | sshd: authentication failed. | 5 | 5760 |
| May 8, 2026 @ 10:44:03.109 | ubuntu_client_1 | sshd: authentication failed. | 5 | 5760 |
| May 8, 2026 @ 10:44:03.109 | ubuntu_client_1 | sshd: authentication failed. | 5 | 5760 |
| May 8, 2026 @ 10:44:01.116 | ubuntu_client_1 | PAM: User login failed. | 5 | 5503 |
| May 8, 2026 @ 10:44:01.115 | ubuntu_client_1 | PAM: User login failed. | 5 | 5503 |
| May 8, 2026 @ 10:44:01.113 | ubuntu_client_1 | PAM: User login failed. | 5 | 5503 |
| May 8, 2026 @ 10:44:01.109 | ubuntu_client_1 | PAM: User login failed. | 5 | 5503 |
| May 8, 2026 @ 10:44:00.097 | ubuntu_client_1 | Suricata: Alert - SSH Brute Force Attempt Detected | 3 | 86601 |
| May 8, 2026 @ 10:43:43.667 | hackcat77vm | Wazuh server started. | 3 | 502 |
| May 8, 2026 @ 10:43:38.641 | hackcat77vm | PAM: Login session closed. | 3 | 5502 |
| May 8, 2026 @ 10:43:34.657 | hackcat77vm | Listened ports status (netstat) changed (new port opened or closed). | 7 | 533 |

**Figure 15** : Détection de la règle custom

Sur la ***figure 15*** on peut voir notre regle custom en troisieme position en partant du haut s'afficher à ***10 :44 :05 :111*** (timestamp) avec le niveau de criticité ***10*** (rule.level) et l'identifiant ***100002*** (rule.id).

On observe également la corrélation entre la détection de la règle au sid ***5760*** (level 5) à multiples reprises avant le déclenchement de notre regle custom (level 10).

# BILAN DES RÈGLES ET VISUALISATIONS

Tableau visualisations :

| Visualisations | Type | Champ principal | Valeur SOC |
|---|---|---|---|
| **Top 10 Alert Types** | Bar | rule.description | Identifier alertes dominantes |
| **Alert Timeline** | Line | timestamp | Détecter anomalies temporelles |
| **Alert Sources** | Pie | rule.groups | Répartition sources logs |
| **Top Attacker IPs** | Table | Data.src_ip | Identifier IPs hostiles |

Tableau règles :

| Rules Ids | Level | Déclenchement | Statut |
|---|---|---|---|
| **100002** | 10 | 5x SID 5760/120s | Détecté |

# CONCLUSION ET AXES D'AMELIORATION

Ce lab m'a permis de développer mes compétences sur l'implémentation d'un dashboard avec Kibana afin de surveiller en temps réel des métriques importantes du réseau. La création de règles wazuh adapté à mon environnement analysant le trafic détecté par le threat hunting.

Dans un SOC les alertes générées ont des niveaux de criticité propre à eux, nous avons pu noter le niveau 3 avec la règle 5760 et le level 10 avec notre règle custom. Plus le niveau est élevé plus l'alerte est critique, dans notre cas la détection d'une seule tentative de connexion par ssh échoué (level 3) peut juste révéler d'un utilisateur ayant mal renseigné son mot de passe mais une répétition abusive (au moins 5 fois en 120 secondes) penche plus vers une tentative d'attaque par brute-force (level 10) d'où un niveau de criticité plus élevé.

Objectifs atteints :

- 4 visualisations opérationnelles
- Dashboard SOC fonctionnel avec filtre Suricata
- Règle brute-force custom validée (level 10 confirmé)
- Pipeline complet : Hydra → sshd → Wazuh → Dashboard

Axes d'amélioration :

- Ajouter une alerte email/Slack quand level >= 10
- Créer un dashboard dédié aux alertes Suricata uniquement
- Intégrer des logs Windows (Event Viewer) — Projet 2.3
- Tester la corrélation multi-sources (Linux + Windows)